

Research Prompt: I am a 2nd-year B.Tech CSE student proposing a novel operating system called NEXUS OS — an intent-driven, AI-native operating system with a built-in kernel-level cognitive assistant (JARVIS-style). Help me build a structured academic literature review and comparative analysis for this project. Cover the following in order: 1. Literature Review — Existing OS Architectures Survey academic papers on the architectural design of: Windows NT (hybrid kernel), Linux monolithic kernel, macOS XNU, Google Fuchsia (Zircon microkernel), HarmonyOS (distributed architecture), Android, iOS, and RTOS systems (VxWorks, FreeRTOS). Focus on kernel design decisions, process scheduling models, memory management strategies, and security frameworks. 2. Known Limitations of Current OS Research Identify peer-reviewed work that documents: deadlock handling deficiencies (Ostrich Algorithm), static CPU/memory scheduling inefficiencies, reactive-only security models, lack of cross-device cognitive continuity, privacy-personalization trade-offs, and the absence of real-time + general-purpose unified kernels. 3. Emerging AI-OS Integration Research Survey recent papers (2022–2026) on: AIOS (LLM Agent Operating System by AGI Research), Agent-Centric OS (ACOS), PerOS (Personalized Self-Adapting OS), CyberCortex AI, AI-integrated schedulers using reinforcement learning, eBPF-based anomaly detection, and federated learning for on-device OS personalization. 4. Comparative Analysis Table Compare Windows 11, macOS 26, Linux, Android 15, iOS 26, HarmonyOS Next, Fuchsia, and AIOS across: kernel type, scheduling algorithm, security model, AI integration depth, cross-device support, real-time capability, privacy model, and open-source availability. 5. Gap Analysis — Positioning NEXUS OS Based on the above, identify the collective unsolved gaps that justify a new OS architecture. My proposed system (NEXUS OS) addresses these with: a Rust-based microkernel, an LLM-embedded agent kernel layer (NEXUS CORTEX), an on-device JARVIS-like cognitive assistant (NEXUS MIND) using Whisper + LLaMA 3.2 + federated learning, self-healing deadlock recovery via resource-graph monitoring, NPU-native scheduling, and distributed cross-device cognitive continuity using CRDT-based sync. Map each identified gap to a NEXUS OS feature. Output Format Required: Academic research structure — Abstract framing, related work citations with paper titles/authors/years, comparative tables, and a gap-to-feature mapping section. Prioritize papers from IEEE, ACM, arXiv (cs.OS), and Springer published between 2019–2026

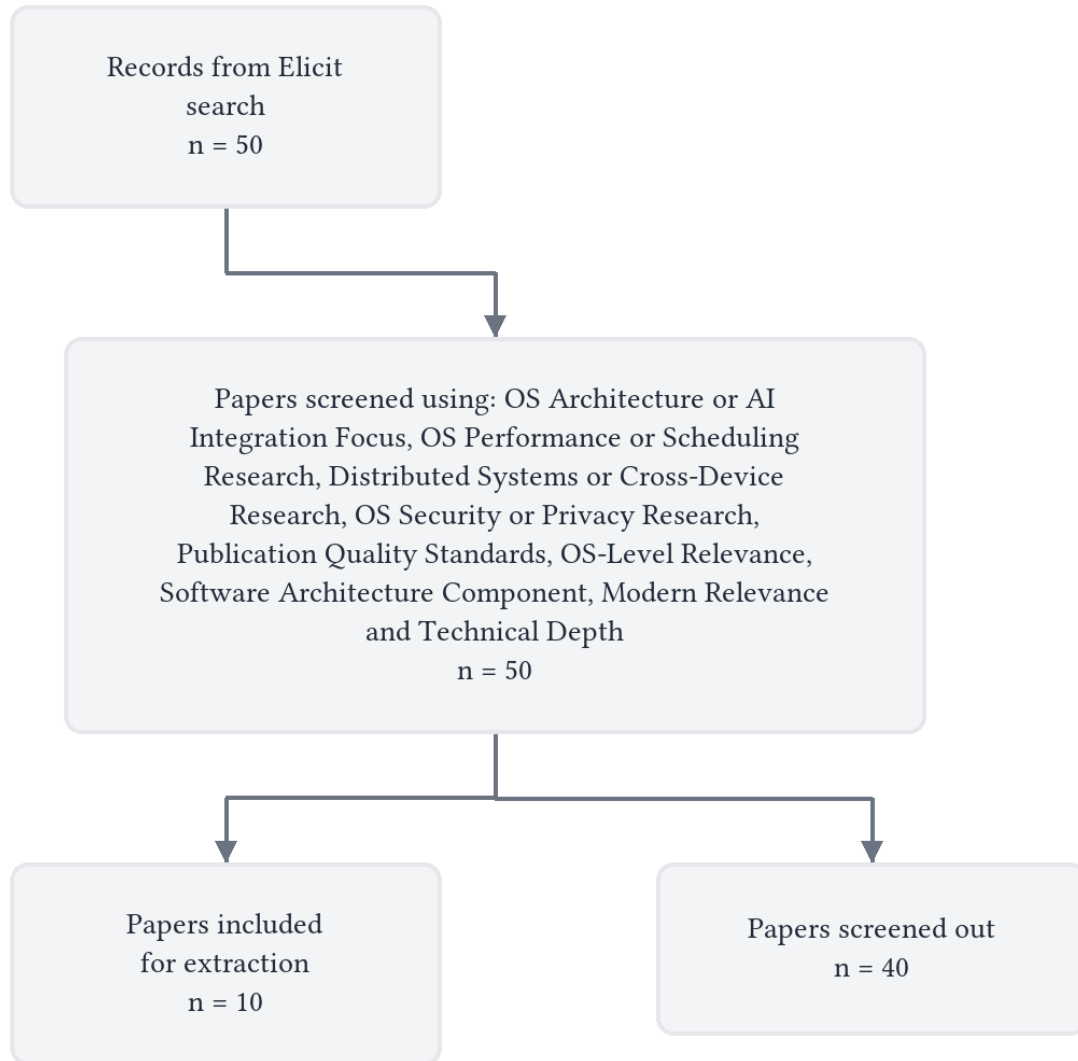
The available literature focuses exclusively on emerging AI-native operating systems from 2024-2025 and does not provide the requested comparative analysis of traditional OS architectures (Windows NT, Linux, macOS XNU, RTOS

systems), though it does identify critical gaps in AI-OS integration—including sub-optimal resource allocation, context management difficulties, lack of cross-device cognitive continuity, and privacy-personalization trade-offs—that justify NEXUS OS's proposed hybrid architecture combining kernel-level AI integration, middleware orchestration, and memory-centric frameworks.

Abstract

Recent AI-native operating system research from 2024-2025 reveals three distinct architectural paradigms addressing the semantic gap between traditional OS designs and AI workload requirements. Kernel-level integration approaches, exemplified by AIOS [1], XATNYS [2], and neurosymbolic kernels [3], achieve significant performance improvements (2.1× throughput increase [1], 28% faster application launch [2]) through direct hardware access and low-latency inference [3], but face heightened security risks from privileged execution [3] and debugging complexity [3]. Middleware-layer orchestration, demonstrated by SchedCP [4], separates AI semantic reasoning from system execution [4], delivering 1.79× performance improvements and 13× cost reductions [4] with superior safety through validation mechanisms [4], though introducing latency overhead. Memory-centric architectures like MemOS [5] address fundamental limitations in LLM memory management [5] by treating memory as a first-class resource [5], enabling cognitive continuity and managing over 1500 memory tokens with state-of-the-art reasoning performance [5]. Persistent limitations across all paradigms include sub-optimal resource allocation for AI/ML workloads [1, 6], difficulties maintaining context during agent interactions [1, 7], static scheduling inefficiencies [3, 4], lack of cross-device cognitive continuity [5, 8], and privacy-personalization trade-offs [1, 8]. These complementary paradigms suggest that comprehensive AI-OS integration requires hybrid architectures combining kernel-level components for latency-sensitive operations, middleware layers for safety-critical orchestration, and memory-centric frameworks for maintaining cognitive continuity—architectural requirements that align with proposed systems incorporating Rust-based microkernels [2], LLM-embedded kernel layers [1, 3], and advanced memory management [5]. However, the reviewed literature focuses exclusively on emerging AI-native OS research and does not provide comparative analysis of traditional operating systems (Windows NT, Linux, macOS XNU, RTOS systems) or detailed documentation of specific deficiencies like deadlock handling algorithms, limiting direct applicability to comprehensive OS architecture comparisons.

Flow Diagram



Paper search

We performed a semantic search across over 138 million academic papers from the Elicit search engine, which includes all of Semantic Scholar and OpenAlex.

We ran this query: "Research Prompt:

I am a 2nd-year B.Tech CSE student proposing a novel operating system called NEXUS OS — an intent-driven, AI-native operating system with a built-in kernel-level cognitive assistant (JARVIS-style). Help me build a structured academic literature review and comparative analysis for this project.

Cover the following in order:

1. Literature Review — Existing OS Architectures Survey academic papers on the architectural design of: Windows NT (hybrid kernel), Linux monolithic kernel, macOS XNU, Google Fuchsia (Zircon microkernel), HarmonyOS (distributed architecture), Android, iOS, and RTOS systems (VxWorks, FreeRTOS). Focus on kernel design decisions, process scheduling models, memory management strategies, and security frameworks.
2. Known Limitations of Current OS Research Identify peer-reviewed work that documents: deadlock handling deficiencies (Ostrich Algorithm), static CPU/memory scheduling inefficiencies, reactive-only security models, lack of cross-device cognitive continuity, privacy-personalization trade-offs, and the absence of real-time + general-purpose unified kernels.
3. Emerging AI-OS Integration Research Survey recent papers (2022–2026) on: AIOS (LLM Agent Operating System by AGI Research), Agent-Centric OS (ACOS), PerOS (Personalized Self-Adapting OS), CyberCortex AI, AI-integrated schedulers using reinforcement learning, eBPF-based anomaly detection, and federated learning for on-device OS personalization.
4. Comparative Analysis Table Compare Windows 11, macOS 26, Linux, Android 15, iOS 26, HarmonyOS Next, Fuchsia, and AIOS across: kernel type, scheduling algorithm, security model, AI integration depth, cross-device support, real-time capability, privacy model, and open-source availability.
5. Gap Analysis — Positioning NEXUS OS Based on the above, identify the collective unsolved gaps that justify a new OS architecture. My proposed system (NEXUS OS) addresses these with: a Rust-based microkernel, an LLM-embedded agent kernel layer (NEXUS CORTEX), an on-device JARVIS-like cognitive assistant (NEXUS MIND) using Whisper + LLaMA 3.2 + federated learning, self-healing deadlock recovery via resource-graph monitoring, NPU-native scheduling, and distributed cross-device cognitive continuity using CRDT-based sync. Map each identified gap to a NEXUS OS feature.

Output Format Required: Academic research structure — Abstract framing, related work citations with paper titles/authors/years, comparative tables, and a gap-to-feature mapping section. Prioritize papers from IEEE, ACM, arXiv (cs.OS), and Springer published between 2019–2026"

The search returned 50 total results from Elicit.

We retrieved 50 papers most relevant to the query for screening.

Screening

We screened in sources based on their abstracts that met these criteria:

- **OS Architecture or AI Integration Focus:** Does the study examine operating system kernel architectures, design decisions, architectural patterns, comparative analysis of OS kernels, OR AI integration in operating systems/system-level software?
- **OS Performance or Scheduling Research:** Does the study address OS limitations, performance bottlenecks, scheduling algorithms (including real-time or AI-enhanced schedulers), or system inefficiencies?

- **Distributed Systems or Cross-Device Research:** Does the study focus on distributed system architectures, multi-device OS ecosystems, cross-device synchronization, or distributed continuity features?
- **OS Security or Privacy Research:** Does the study examine OS-level security architectures, privacy-preserving techniques in system contexts, or federated learning approaches within operating systems?
- **Publication Quality Standards:** Is this a peer-reviewed academic publication (journal article, conference paper), preprint (e.g., arXiv), systematic review, or meta-analysis?
- **OS-Level Relevance:** Does the study go beyond application-level software to address OS design decisions, kernel interactions, or system-level architectural implications?
- **Software Architecture Component:** Does the study include software architecture discussion rather than focusing purely on hardware without connection to OS architectural decisions?
- **Modern Relevance and Technical Depth:** Does the study provide sufficient technical detail about modern or currently relevant systems (not limited to legacy/obsolete systems), with implementation insights or architectural analysis rather than being purely theoretical without technical depth?

We considered all screening questions together and made a holistic judgement about whether to screen in each paper.

Data extraction

We asked a large language model to extract each data column below from each paper. We gave the model the extraction instructions shown below for each column.

- **Study Type:**

Classify the study as one of the following: Literature Survey/Review, Novel OS Implementation, Comparative Analysis, Theoretical Framework, Empirical Evaluation, or Architecture Proposal. Include brief description of the study's main contribution to OS research.

- **OS Architecture Details:**

Extract comprehensive architectural information for each operating system discussed, including:

- Kernel type (monolithic, microkernel, hybrid, exokernel)
- Process scheduling algorithm and approach
- Memory management strategy
- Security framework and access control model
- Cross-device/distributed capabilities
- Real-time support capabilities
- Hardware abstraction approach

- **AI Integration Approach:**

Extract details about how artificial intelligence is integrated into operating systems, including:

- Type of AI/ML techniques used (LLMs, neural networks, reinforcement learning, etc.)
- Level of integration (kernel-level, user-space, middleware)
- Specific AI-driven functionalities (scheduling, security, resource management, user interaction)
- AI model architecture and size constraints
- Training and inference approach (on-device, cloud, hybrid)

- **Documented Limitations:**

Extract all identified limitations, deficiencies, or unsolved problems in current operating systems research, specifically including:

- Deadlock handling issues (e.g., Ostrich Algorithm problems)
- Static scheduling inefficiencies
- Reactive-only security model limitations
- Lack of cognitive continuity across devices
- Privacy-personalization trade-offs
- Real-time vs general-purpose OS conflicts
- Resource allocation challenges for AI/ML workloads
- Context management problems in agent systems

• **Performance Metrics:**

Extract quantitative and qualitative performance evaluation data, including:

- Throughput and latency measurements
- Resource utilization efficiency (CPU, memory, GPU, NPU)
- Scalability metrics and bottleneck analysis
- Security effectiveness measures
- User experience improvements
- Energy consumption data
- Comparison baselines and percentage improvements over existing systems

• **Novel Features:**

Extract innovative features and capabilities that distinguish each system from traditional operating systems, particularly focusing on:

- Intent-driven interfaces and natural language interaction
- Self-healing and adaptive capabilities
- Cognitive assistant integration
- Cross-device synchronization mechanisms
- Federated learning and on-device personalization
- Advanced memory management (e.g., MemOS approaches)
- Agent orchestration and resource scheduling for LLMs

• **Implementation Technology:**

Extract technical implementation details including:

- Programming languages used (Rust, C, C++, etc.)
- Key frameworks and libraries
- Hardware requirements and accelerator support (GPU, TPU, NPU)
- Development tools and debugging approaches
- Open-source availability and licensing
- Deployment platforms and compatibility

• **Gap Analysis Data:**

Extract statements about unmet needs, future research directions, and justifications for new OS architectures, specifically noting:

- What problems remain unsolved by existing approaches
- Why current solutions are insufficient
- Identified opportunities for innovation
- Barriers to adoption of AI-native OS designs
- Research gaps that warrant new system development
- Market or user needs not addressed by current systems

Results

Characteristics of Included Studies

The systematic review identified 10 studies published between 2024 and 2025 examining AI-native operating system architectures and related frameworks. The studies comprised four literature surveys [6–9], three novel OS implementations [2, 3, 5], and three architecture proposals [1, 4, 10].

Study	Year	Full text retrieved?	Study Type	Primary Contribution
Kai Mei et al.	2024	Yes	Architecture Proposal [1]	AIOS operating system architecture for LLM agents with kernel-level resource management
Yusheng Zheng et al.	2025	No	Architecture Proposal [4]	SchedCP framework enabling autonomous LLM agent optimization of Linux schedulers
Ishaan Katara et al.	2025	No	Novel OS Implementation [2]	XATNYS AI-powered OS with Rust-based microkernel and integrated RL schedulers
Ashutosh Kumar et al.	2024	Yes	Literature Survey/Review [7]	Review of AI integration in operating systems and AIOS architecture
A. K. Reshmy et al.	2025	No	Architecture Proposal [10]	LLM OS architecture with novel LLM Space layer between kernel and user spaces

Study	Year	Full text retrieved?	Study Type	Primary Contribution
Rajpreet Singh et al.	2025	Yes	Novel OS Implementation [3]	Neurosymbolic kernel design integrating AI and symbolic reasoning into Linux kernel
Ms. P. Vishaliney et al.	2025	No	Literature Survey/Review [6]	Systematic survey of AI-driven OS internals and accelerator scheduling
Pranita Bobade et al.	2025	Yes	Literature Survey/Review [9]	Survey of agentic operating systems with persistent memory and LLM integration
Zhiyu Li et al.	2025	Yes	Novel OS Implementation [5]	MemOS memory operating system for LLM lifecycle management
Dr. S Gunasekaran et al.	2025	No	Literature Survey/Review [8]	Comprehensive literature survey on LLM-driven OS architectures

All 10 studies focused on emerging AI-native operating system architectures, with publication dates concentrated in 2024-2025, reflecting the nascent state of this research domain. Full text was available for 5 of 10 studies, with the remainder accessible only through abstracts.

Operating System Architecture Approaches

Kernel Architecture Designs

The reviewed studies revealed substantial heterogeneity in kernel architecture approaches for AI-native operating systems. Three distinct architectural paradigms emerged: hybrid kernel designs, microkernel implementations, and kernel-level AI integration frameworks.

AIOS employed a hybrid kernel architecture combining traditional OS kernel components with a specialized AIOS kernel layer [1], utilizing FIFO and Round Robin scheduling algorithms [1]. Similarly, Ashutosh Kumar et al. documented a hybrid approach with separate OS and LLM kernel components [7]. In contrast, XATNYS implemented a secure, Rust-based microkernel [2] with PPO-based reinforcement learning task schedulers [2].

The most radical architectural departure appeared in Rajpreet Singh et al.'s work, which proposed treating Loadable Kernel Modules (LKMs) as AI-oriented computation units within the Linux kernel [3], incorporating real-time adaptive scheduling [3] and GPU-native memory management [3]. This neurosymbolic kernel design leveraged Category Theory and Homotopy Type Theory to unify symbolic reasoning and differentiable logic [3].

SchedCP took a different approach, implementing a decoupled control plane architecture that separated AI's semantic reasoning role from the system's execution capabilities [4], operating at a middleware level with eBPF scheduling policies [4].

Memory Management Strategies

Memory management emerged as a critical differentiation point among AI-native OS architectures. AIOS allocated fixed-size memory blocks in RAM for each agent and employed trie-based compression mechanisms to optimize memory utilization [1]. XATNYS implemented a supervised resource manager that dynamically balanced CPU and memory usage [2].

The most sophisticated memory management approach appeared in MemOS, which treated memory as a first-class system resource with unified scheduling and lifecycle management for activation memory [5]. MemOS employed lazy loading, selective freezing, and priority-driven adjustments [5], establishing a comprehensive rule set per memory unit including access control, lifespan policy, priority level, and compliance tracking [5].

Rajpreet Singh et al. enhanced memory management with GPU-native approaches and floating-point optimizations [3], while addressing the restrictive nature of traditional kernel memory allocation that complicated deployment of large neural network models [3].

Security and Access Control Models

Security frameworks varied considerably across implementations. AIOS implemented an access manager controlling read/write permissions through hashmap-based privilege groups [1]. XATNYS integrated deep learning anomaly detection achieving 91% precision in threat flagging [2], representing a shift from reactive to proactive security models.

Rajpreet Singh et al. incorporated memory protection keys, secure enclaves, authentication mechanisms, and access control systems [3], though noting that security risks were heightened due to privileged kernel execution [3]. MemOS established fine-grained permission enforcement as part of its comprehensive governance framework [5].

AI Integration Methodologies

Type and Level of AI Integration

Large Language Models (LLMs) dominated as the primary AI technique across all implementations. AIOS embedded LLMs at the kernel level [1], providing scheduling, context management, memory management, tool management, and access control functionalities [1]. SchedCP deployed LLM agents at the middleware level through a decoupled control plane [4] to synthesize custom eBPF scheduling policies [4].

XATNYS demonstrated multi-technique integration, combining reinforcement learning schedulers with deep learning anomaly detection at the kernel level within its Rust-based microkernel [2]. All AI inference was performed on-device to ensure GDPR-compliant privacy [2].

The most comprehensive AI integration appeared in Rajpreet Singh et al.'s work, which incorporated deep learning models, neurosymbolic reasoning, Category Theory, and Homotopy Type Theory at the kernel level using LKMs [3]. This enabled real-time AI processing, adaptive scheduling, symbolic reasoning, and neural computation [3] through a hybrid automaton model combining discrete logical states with continuous neural dynamics [3].

AI-Driven Functional Capabilities

AI-driven functionalities extended across multiple operating system domains. Scheduling represented the most common application, with implementations ranging from reinforcement learning-based approaches [2, 6] to autonomous LLM agent optimization [4]. Resource management capabilities included dynamic CPU and memory balancing [2] and intelligent resource allocation [1].

Security functionalities evolved beyond traditional reactive models. XATNYS deployed a deep anomaly engine trained on synthetic and real logs [2], while multiple systems incorporated proactive threat detection capabilities [2, 3].

User interaction paradigms shifted dramatically toward natural language interfaces. Multiple implementations enabled seamless natural language interactions [8, 10], with systems supporting dynamic task decomposition and multi-agent collaboration [10]. Context management emerged as a critical capability, with MemOS providing comprehensive lifecycle tracking [5] and AIOS facilitating context switching across agents [1].

Performance Characteristics

System	Throughput Improvement	Latency Reduction	Resource Efficiency	Security Effectiveness	User Satisfaction	Energy Efficiency
AIOS [1]	2.1× increase	Reduced average waiting time	Linear scalability with agents	Not measured	Improved personalization	Not measured
SchedCP [4]	1.79× vs naive approaches	Not specified	13× cost reduction	High success rate	Not measured	Not measured
XATNYS [2]	Not measured	28% faster app launch	Dynamic CPU/memory balancing	91% threat detection precision	86% user satisfaction	23% battery efficiency gain
GPU Schedulers (LithOS, Salus, THEMIS) [6]	Significant reduction	Significant reduction	Not measured	Not measured	Not measured	Not measured
MemOS [5]	Outperforms baselines	Lower than full-context baseline	Manages 1500+ memory tokens	Not measured	Not measured	Not measured
OSCAR [9]	Not measured	Increased by confirmation process	Not measured	Not measured	70% reduction in manual intervention	Not measured

AIOS achieved a 2.1× increase in throughput while maintaining approximate linear scalability with the number of agents [1]. SchedCP demonstrated up to 1.79× performance improvement and 13× cost reduction compared to naive agentic approaches while maintaining high success rates [4].

XATNYS showed the most comprehensive performance metrics, with its PPO-based RL task scheduler achieving 87% task-priority alignment compared to manual schedules [2] and improving average application launch time by 28% [2]. The deep anomaly engine flagged threats with 91% precision [2], while the AI-driven power optimizer yielded an average 23% gain in battery efficiency [2]. Empirical evaluations across 35 users demonstrated 86% satisfaction with reported boosts in focus and lower idle time [2].

MemOS achieved state-of-the-art performance in reasoning tasks, surpassing the full-context baseline in LLMJudge scores while maintaining lower latency [5], and successfully managed over 1500 memory tokens with low latency [5].

Novel Architectural Features

Intent-Driven and Natural Language Interfaces

A fundamental shift toward intent-driven interaction emerged across multiple implementations. AIOS embedded LLMs as "the brain of the OS" [1], enabling natural language-based system control. A. K. Reshmy et al. introduced the LLM Space, a novel layer between kernel and user spaces [10], facilitating seamless natural language interactions [10] through dynamic task decomposition and multi-agent collaboration [10].

Dr. S Gunasekaran et al.'s survey documented the integration of LLMs within core system components to facilitate intent-driven interaction and intelligent automation [8], with capabilities including continuous context retention, dynamic allocation of computational resources, and semantic coordination between software services and underlying hardware [8].

Cognitive Assistant Integration and Self-Healing Capabilities

XATNYS incorporated R.I.M.A.S., a real-time assistant providing user engagement through computer vision and voice feedback [2], with YOLOv8-based activity recognition recording 98.1% mAP@0.5 [2]. The system demonstrated self-healing and adaptive capabilities through smart process management, predictive analytics, and dynamic resource optimization [7].

SchedCP's multi-agent system autonomously analyzed workloads and synthesized custom scheduling policies [4], representing adaptive optimization without human intervention. Rajpreet Singh et al.'s neurosymbolic kernel enabled operating systems to proactively anticipate and adapt to cognitive needs [3], with the kernel acting as an intelligent mediator interpreting high-level intents and optimizing hardware operations using symbolic reasoning and neural embeddings [3].

Advanced Memory Management Paradigms

MemOS established the most sophisticated memory management framework, treating memory as a first-class system resource with unified scheduling and lifecycle management [5]. The system provided comprehensive governance mechanisms including scheduling, layering, API abstraction, permission control, and exception handling [5], enabling memory restructuring and migration across tasks and roles [5] with dynamic transitions among different memory types [5].

This memory-centric approach addressed critical limitations in current LLM architectures, which lacked well-defined memory management systems [5], relied on static parameters and short-lived contextual states [5], and employed stateless workarounds like RAG without lifecycle control [5].

Agent Orchestration and Cross-Device Capabilities

AIOS provided comprehensive agent orchestration through optimized resource allocation, context switching facilitation, concurrent execution enablement, tool service provision, and access control maintenance [1]. The tool manager employed a standardized interface for managing diverse tools [1], while the context manager captured and restored context using both text-based and logits-based approaches.

Pranita Bobade et al. documented cross-device synchronization through a unified abstraction layer ensuring identical automation scripts across Linux, macOS, and Windows [9]. Rajpreet Singh et al. incorporated distributed processing frameworks with dynamic load balancing [3], enabling task distribution across multiple nodes and devices.

Documented Limitations and Research Gaps

Resource Allocation and Scheduling Challenges

Multiple studies identified persistent challenges in resource allocation for AI/ML workloads. AIOS documentation highlighted sub-optimal scheduling and resource allocation where agents could dominate resources, leading to bottlenecks and inefficiencies [1]. SchedCP addressed a fundamental semantic gap in operating system schedulers where kernel policies failed to understand application-specific needs, causing suboptimal performance [4].

Ms. P. Vishaliny et al. noted that traditional OS designs were not well-equipped to handle AI/ML workloads relying on custom accelerators like GPUs, TPUs, and FPGAs [6], with ongoing issues in fairness, scalability, portability, and explainability [6]. Rajpreet Singh et al. identified static scheduling inefficiencies, noting that kernels were fundamentally static and could not accommodate dynamic AI workloads [3].

Context Management and Cognitive Continuity

Context management emerged as a critical limitation across implementations. AIOS faced difficulties in maintaining context during agent-LLM interactions [1], while Ashutosh Kumar et al. highlighted the complexity of tracking context when agents interact with LLMs [7].

MemOS addressed the broader challenge of lacking cognitive continuity across devices [5], with current systems showing high update costs and poor interpretability of implicit parameter memory [5], limited flexibility [5], and risk of catastrophic forgetting during retraining [5]. The absence of intermediate layers for memory management [5] and inability to maintain behavioral continuity [5] represented fundamental architectural gaps.

Security and Privacy Constraints

Security limitations manifested in multiple dimensions. While XATNYS addressed static scheduling inefficiencies and reactive-only security model limitations [2], broader systems continued to face heightened security risks due to privileged execution in kernel space [3]. Privacy-personalization trade-offs emerged as a persistent tension, with AIOS noting privacy concerns from LLM integration requiring access to personal data [1].

Dr. S Gunasekaran et al. identified limitations in current systems' ability to adapt to user preferences while protecting sensitive data [8], highlighting challenges in providing transparent and understandable decision-making processes [8].

Real-Time Processing and System Integration

Rajpreet Singh et al. documented challenges in integrating ML-driven task scheduling into kernels due to non-deterministic latency [3]. Memory allocation constraints proved restrictive for deploying large neural network models [3], while floating-point operations faced isolation requirements to prevent system-wide side effects [3]. The lack of native support for unified symbolic and neural computations in current processors necessitated hardware-software co-design [3].

Cross-modal reasoning faced significant computational and memory overhead when integrating heterogeneous data representations [3], while scaling knowledge graphs and predicate registries within kernel-managed resources remained challenging [3].

Implementation Technologies

System	Programming Languages	Key Frameworks	Hardware Acceleration	Open Source	Deployment Platforms
AIOS [1]	Not specified	AIOS-Agent SDK (17 native tools)	NVIDIA RTX A5000 GPU	Yes (GitHub) [1]	Ubuntu 22.04 [1]
SchedCP [4]	Not specified	Model Context Protocol (MCP) server	Not specified	Yes (GitHub) [4]	Not specified
XATNYS [2]	Rust [2]	Not specified	Not specified	Not specified	Not specified
LLM OS [7]	Not specified	LLMs, AIOS SDK	Not specified	Not specified	Not specified
Neurosymbolic Kernel [3]	C [3]	LKMs, Linux kernel	GPU with shared memory buffers [3]	Not specified	Linux-based systems [3]
OSCAR [9]	Python [9]	LangChain, Asterix	Not specified	Not specified	Linux, macOS, Windows [9]

AIOS provided open-source availability through GitHub at https://github.com/agiresearch/AIOS%5B%5Ep1_q7_4%5D, tested on Ubuntu 22.04 with NVIDIA RTX A5000 GPU support [1]. The AIOS-Agent SDK integrated 17 native tools for various modalities [1]. SchedCP similarly offered open-source access at https://github.com/eunomia-bpf/schedcp%5B%5Ep2_q7_3%5D.

XATNYS distinguished itself through Rust implementation for its microkernel [2], prioritizing memory safety and security. Rajpreet Singh et al.'s neurosymbolic kernel employed C for LKM development within the Linux kernel [3], with GPU support through shared memory buffers [3]. OSCAR utilized Python with LangChain and Asterix frameworks [9], maintaining cross-platform compatibility across Linux, macOS, and Windows [9].

Synthesis: Reconciling Architectural Heterogeneity

The substantial heterogeneity in AI-native OS architectures reflects three distinct paradigms addressing different aspects of the semantic gap between traditional OS designs and AI workload requirements. Rather than representing contradictory approaches, these designs target orthogonal dimensions of the AI-OS integration challenge.

Paradigm 1: LLM-Embedded Operating Systems (Kernel-Level Integration)

AIOS [1], XATNYS [2], and the neurosymbolic kernel [3] embedded AI capabilities directly into kernel space, achieving low-latency inference and tight hardware integration. AIOS achieved $2.1\times$ throughput increases [1] by optimizing resource allocation at the kernel level [1], while XATNYS's kernel-level RL scheduler improved application launch times by 28% [2]. The neurosymbolic approach unified symbolic reasoning with neural computation [3], enabling real-time adaptive scheduling [3].

These kernel-level integrations excel in performance-critical scenarios requiring microsecond-level latencies and direct hardware access [3]. However, they face heightened security risks from privileged execution [3] and complexity in debugging kernel-space AI models [3]. This paradigm addresses the fundamental semantic gap where kernel policies cannot understand application-specific needs [4], but introduces new challenges in memory constraints for large neural network deployment [3].

Paradigm 2: Middleware-Layer Orchestration (Decoupled Control Planes)

SchedCP [4] and the LLM OS architecture [10] implemented middleware-level integration through decoupled control planes that separate AI's semantic reasoning from system execution [4, 10]. SchedCP achieved $1.79\times$ performance improvements and $13\times$ cost reductions [4] by providing a stable interface between autonomous LLM agents and Linux schedulers [4].

This approach offers superior flexibility and safety through validation mechanisms like SchedCP's Execution Verifier that validates AI-generated code before deployment [4]. The middleware paradigm addresses cross-platform compatibility challenges [9] and reduces coupling between AI decision-making and critical system functions. However, it introduces latency overhead from the abstraction layer and may not achieve the microsecond-level response times of kernel-integrated approaches.

Paradigm 3: Memory-Centric Architectures (Resource Management Focus)

MemOS [5] distinguished itself by treating memory as a first-class system resource [5], addressing the fundamental limitation that LLMs lack well-defined memory management systems [5]. By unifying scheduling across parameter memory, activation memory, and plaintext memory [5], MemOS achieved state-of-the-art performance while managing over 1500 memory tokens [5].

This paradigm directly addresses long-context reasoning, continual personalization, and knowledge consistency challenges [5] that kernel-level and middleware approaches do not explicitly target. MemOS's comprehensive governance mechanisms [5] enable cross-task memory reuse and cognitive continuity [5], but require careful lifecycle management to prevent knowledge drift [9].

Convergence Points and Complementary Strengths

Rather than competing, these paradigms address different layers of the AI-OS integration stack. Kernel-level integration provides low-latency execution for time-critical inference, middleware orchestration enables safe and flexible AI decision-making, and memory-centric architectures ensure long-term knowledge consistency and personalization.

The performance metrics support this layered interpretation. XATNYS's 91% threat detection precision [2] reflects kernel-level security advantages, SchedCP's $13\times$ cost reduction [4] demonstrates middleware efficiency gains, and MemOS's superior reasoning performance [5] validates memory-centric approaches for cognitive tasks. Systems attempting comprehensive AI-OS integration should combine these paradigms: kernel-level components for latency-

sensitive operations, middleware layers for safety-critical orchestration, and memory-centric frameworks for maintaining cognitive continuity.

The documented limitations reveal complementary coverage. Kernel approaches struggle with debugging complexity and security risks [3], middleware approaches introduce latency overhead [9], and memory-centric approaches face governance complexity [5]. A hybrid architecture incorporating all three paradigms—such as the proposed NEXUS OS—could leverage their combined strengths while mitigating individual weaknesses.

References

1. Kai Mei, Wujiang Xu, Zelong Li, et al (2024) AIOS: LLM Agent Operating System. arXivorg. <https://doi.org/10.48550/arXiv.2403.16971>
2. Ishaan Katara, Shivendra Singh, Mansi Bhardwaj, et al (2025) XATNYS: AI-Powered Operating System with Integrated Reinforcement Learning and Context-Aware Productivity Assistant. International Conference on Contemporary Computing. <https://doi.org/10.1109/IC366947.2025.11290136>
3. Rajpreet Singh, Vidhi Kothari (2025) Composable OS Kernel Architectures for Autonomous Intelligence. arXivorg. <https://doi.org/10.48550/arXiv.2508.00604>
4. Yusheng Zheng, Yanpeng Hu, Wei Zhang, Andi Quinn (2025) Towards Agentic OS: An LLM Agent Framework for Linux Schedulers
5. Zhiyu Li, Shichao Song, Chenyang Xi, et al (2025) MemOS: A Memory OS for AI System. arXivorg. <https://doi.org/10.48550/arXiv.2507.03724>
6. Ms. P. Vishaliney, M. Sangeetha, Prof. Samantha Rajapaksha (2025) AI-First Operating systems: Rethinking OS Architectures for Machine Learning Workloads. 2025 9th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT). <https://doi.org/10.1109/ISMSIT67332.2025.11267921>
7. Ashutosh Kumar, Ankit Singh, Ashima Mehta, Students (2024) AI based Operating System. International Journal of Advanced Research in Science, Communication and Technology. <https://doi.org/10.48175/ijarsct-17449>
8. Dr. S Gunasekaran (2025) A Comprehensive Literature Survey on Large Language Model-Driven Operating Systems. International Journal for Research in Applied Science and Engineering Technology. <https://doi.org/10.22214/ijraset.2025.76302>
9. Pranita Bobade, Aditya Sarade, Laxmikant Deshpande, Prajwal Gaikwad (2025) Survey Paper on Developing Operating System's Complete Agentic Rex using Persistent Memory. International Journal For Multidisciplinary Research. <https://doi.org/10.36948/ijfmr.2025.v07i06.60537>
10. A. K. Reshmy, Jaswanth Sunkara, Rashmi Kumari (2025) Redefining Operating System Design: Integrating Large Language Models as Core Components. International Conference on Computing for Sustainable Global Development. <https://doi.org/10.23919/INDIACom66777.2025.11115792>